

designing state

Daniel Jackson

what you'll learn today

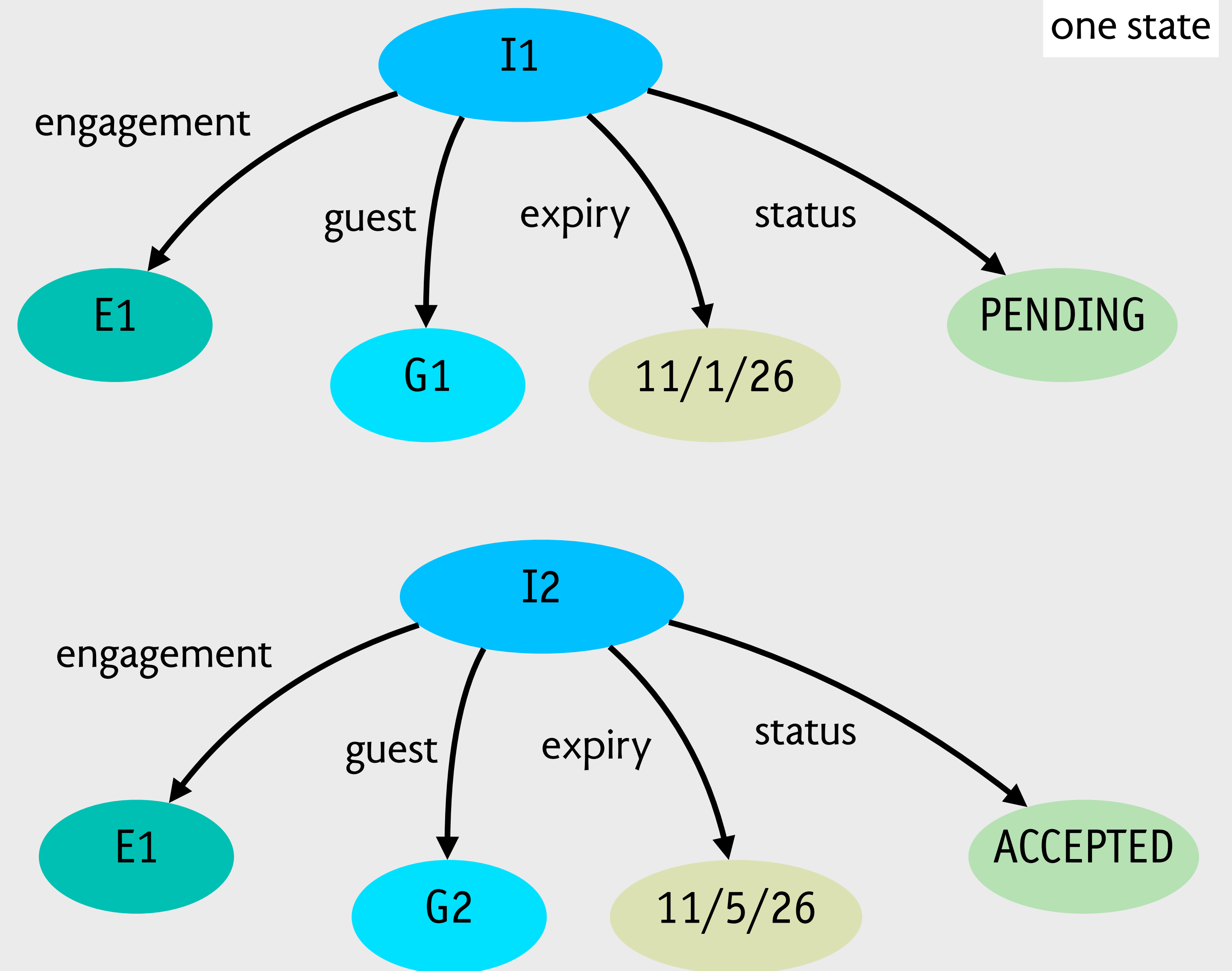
view structuring: what objects can't do
trickier concepts with richer states

*how state is
structured*

visualizing a state

state

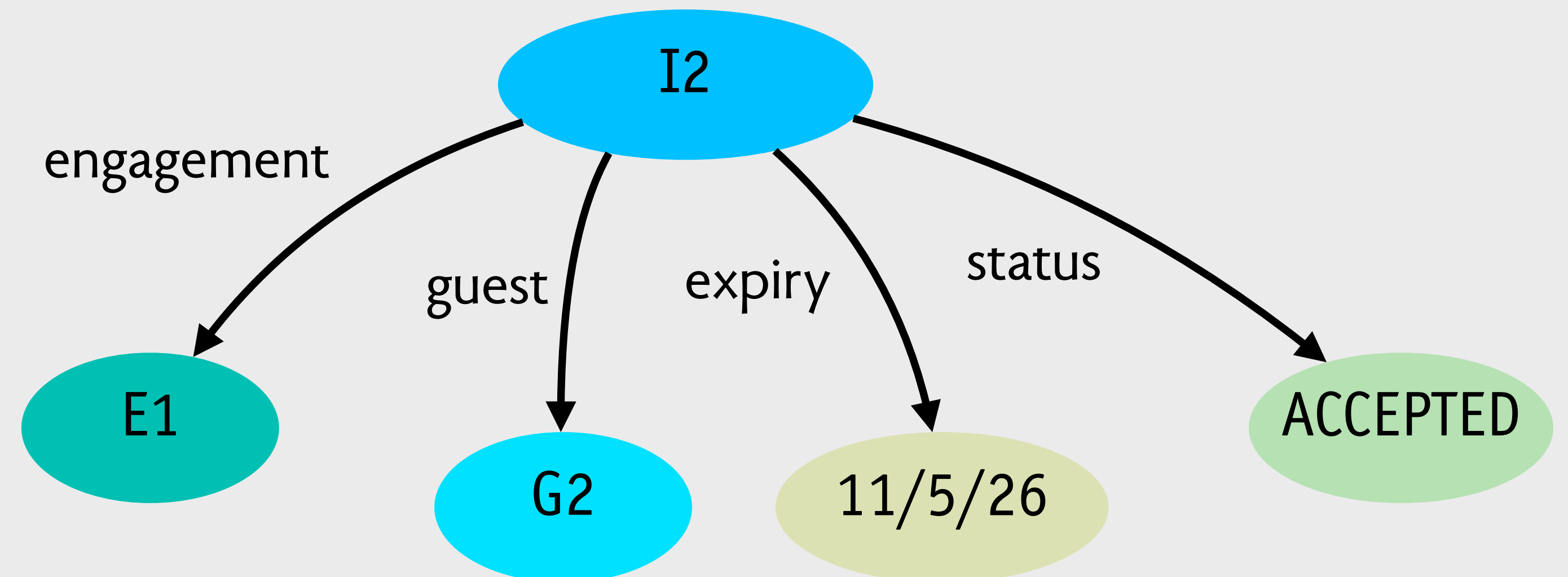
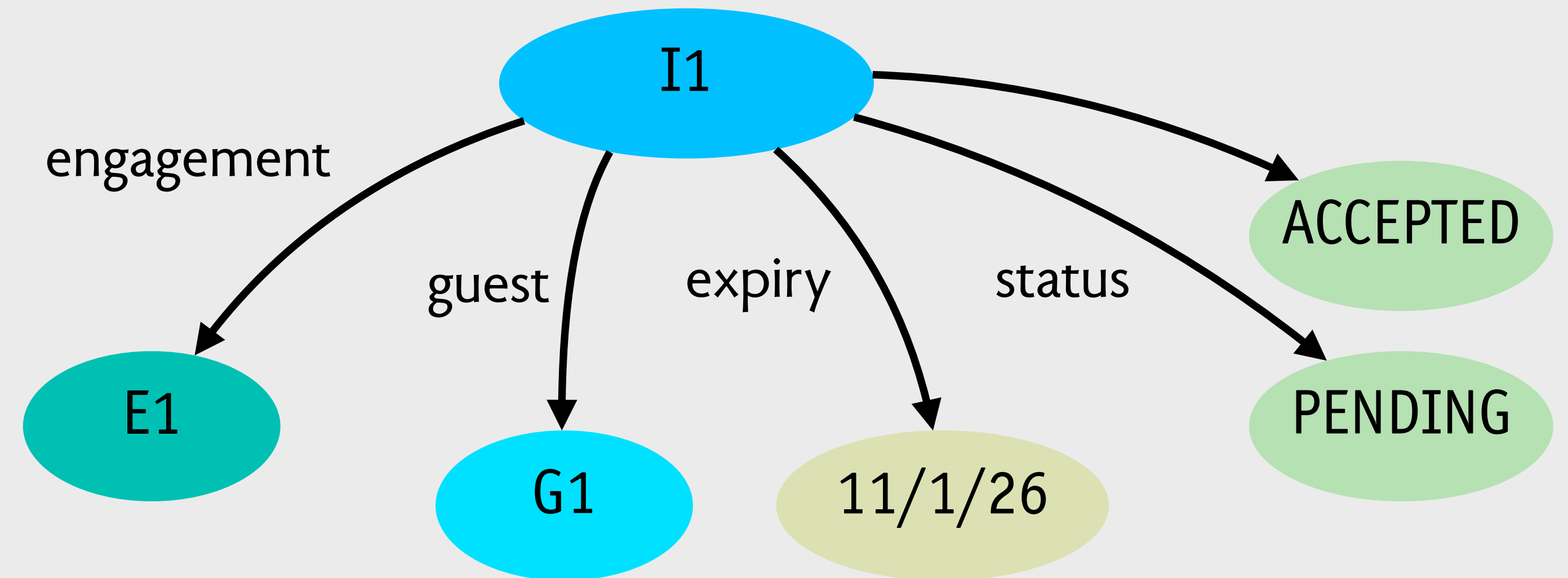
a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED



visualizing a transition

accept (invitation: Invitation)
where invitation has status PENDING
then set status of invitation to ACCEPTED

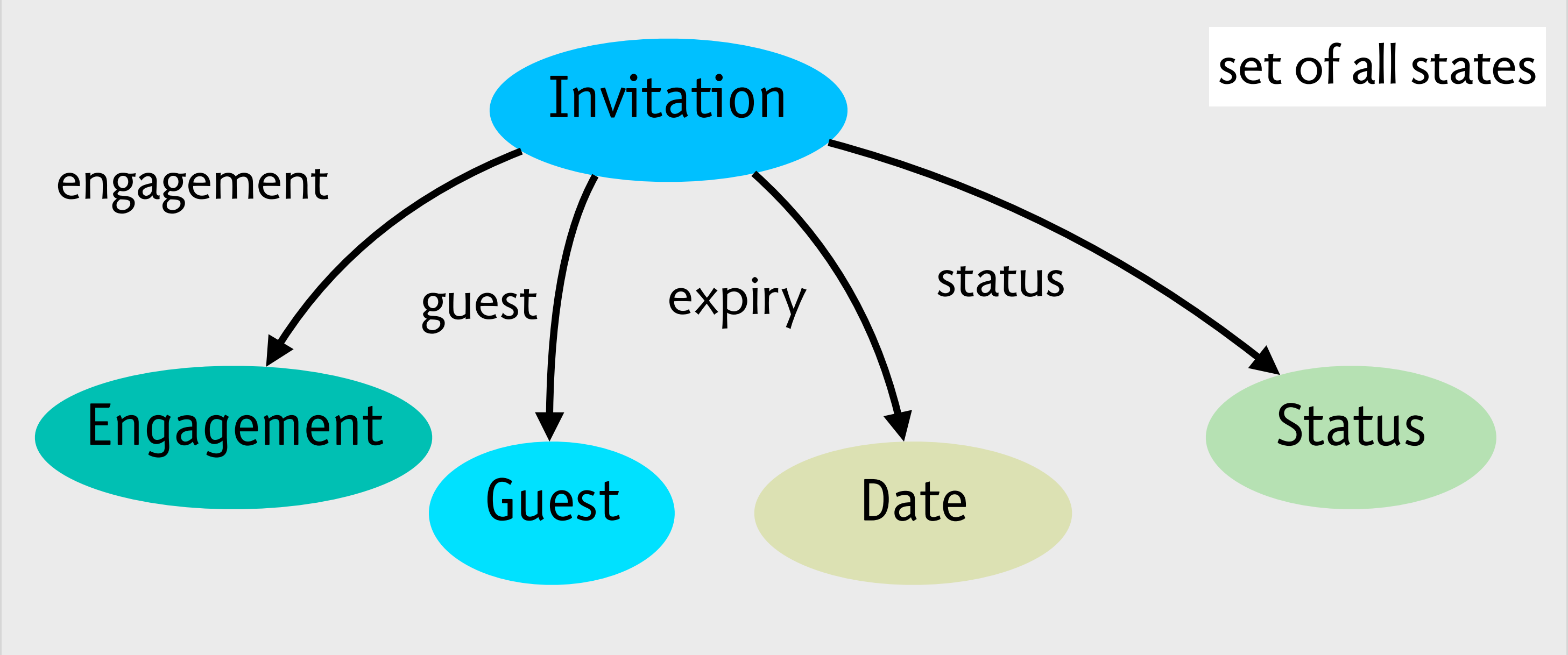
accept (I1)



visualizing the set of all states

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

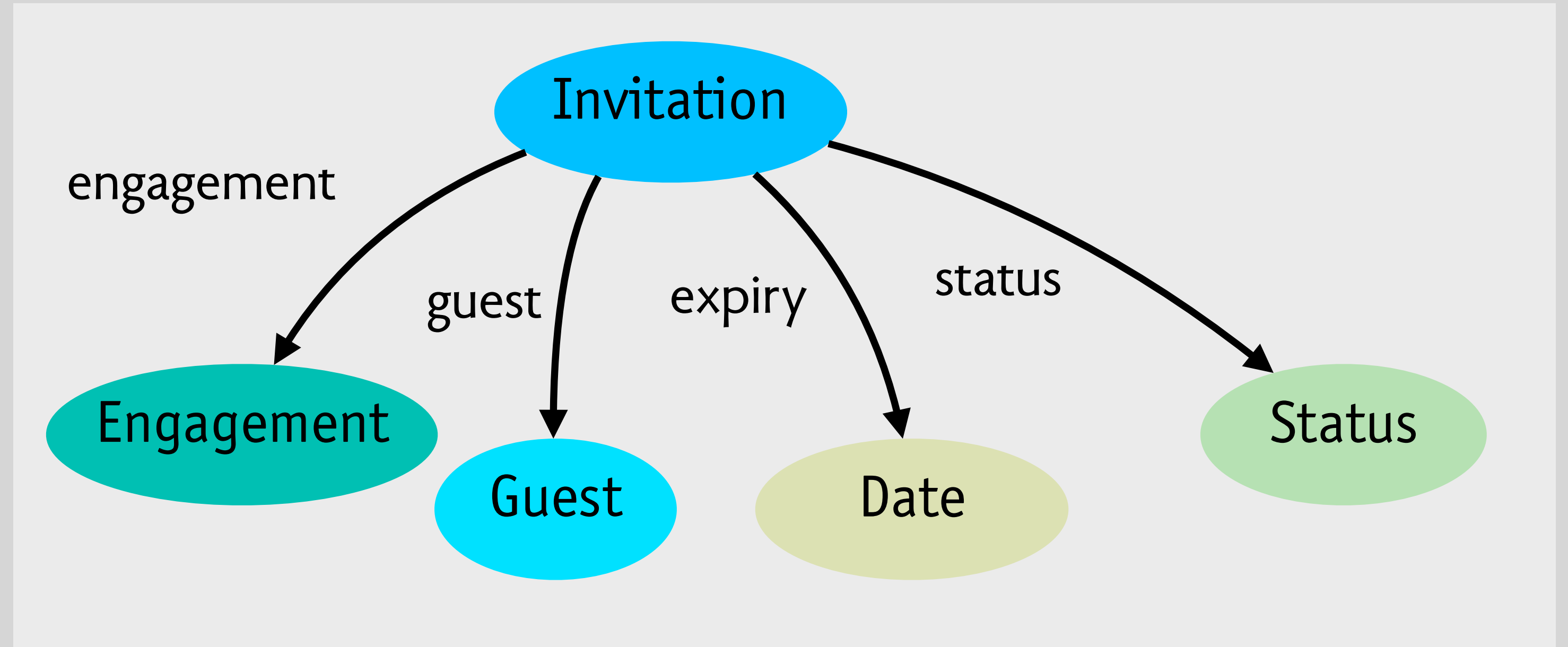


entity-relationship diagram

what are these things?

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED



Engagement

Guest

Invitation

these are entities or individuals

Date

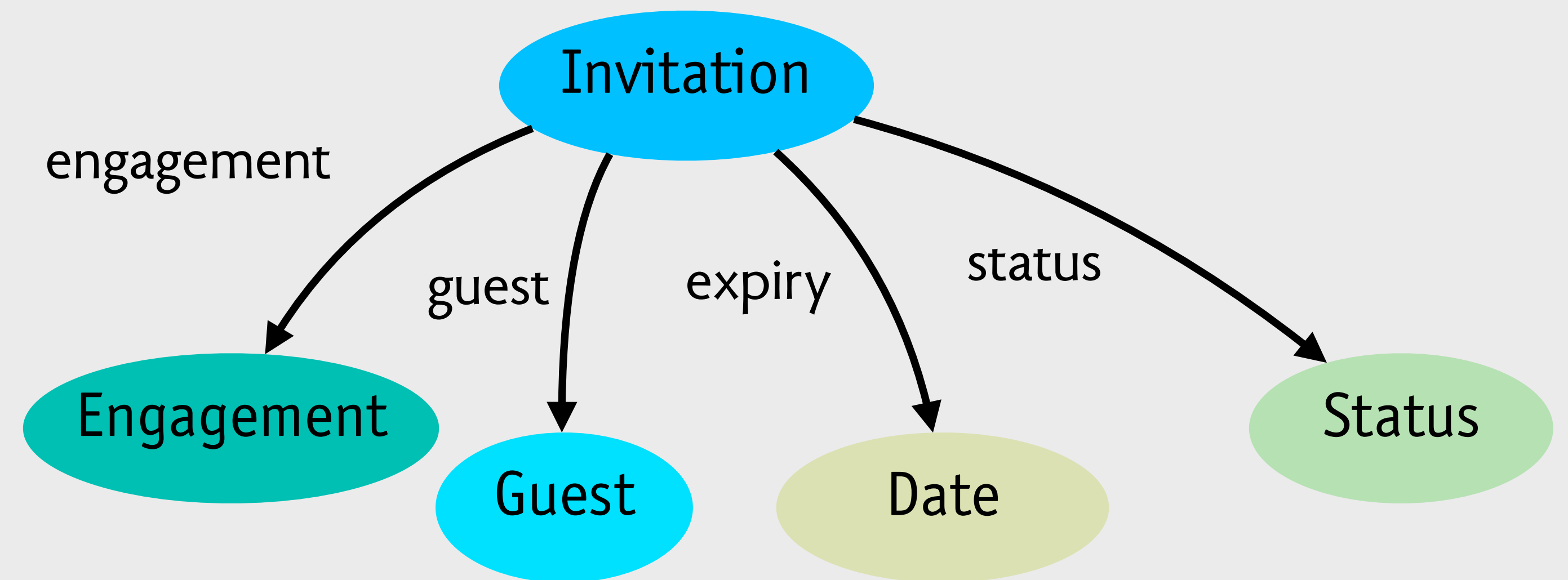
Status

these are values

individuals are not objects

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

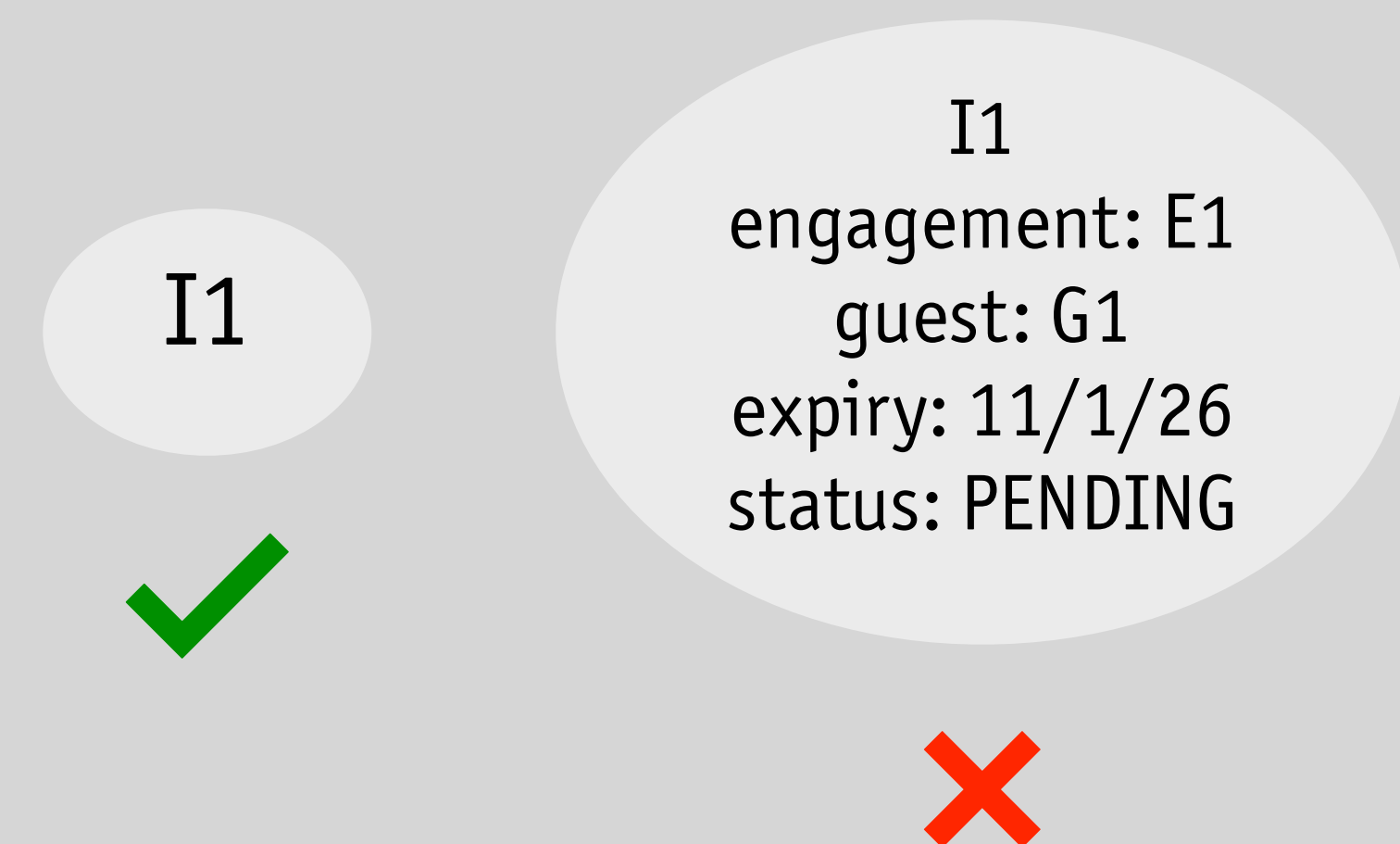


invite (G1, E1): (I1)

what is the invitation you received?
just an identifier or "handle"!

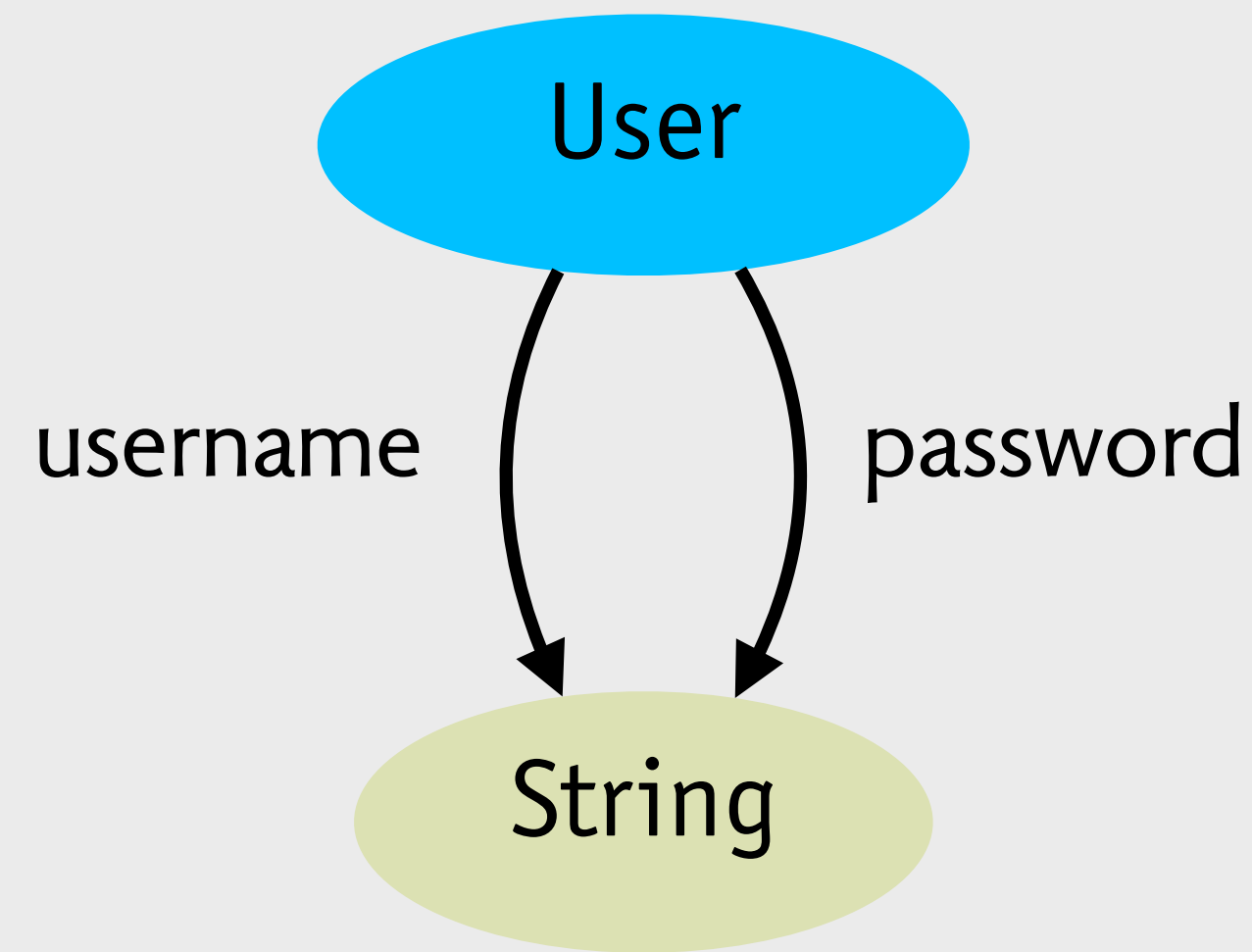
accept (I1)

what is the invitation you sent back?
just an identifier or "handle"!

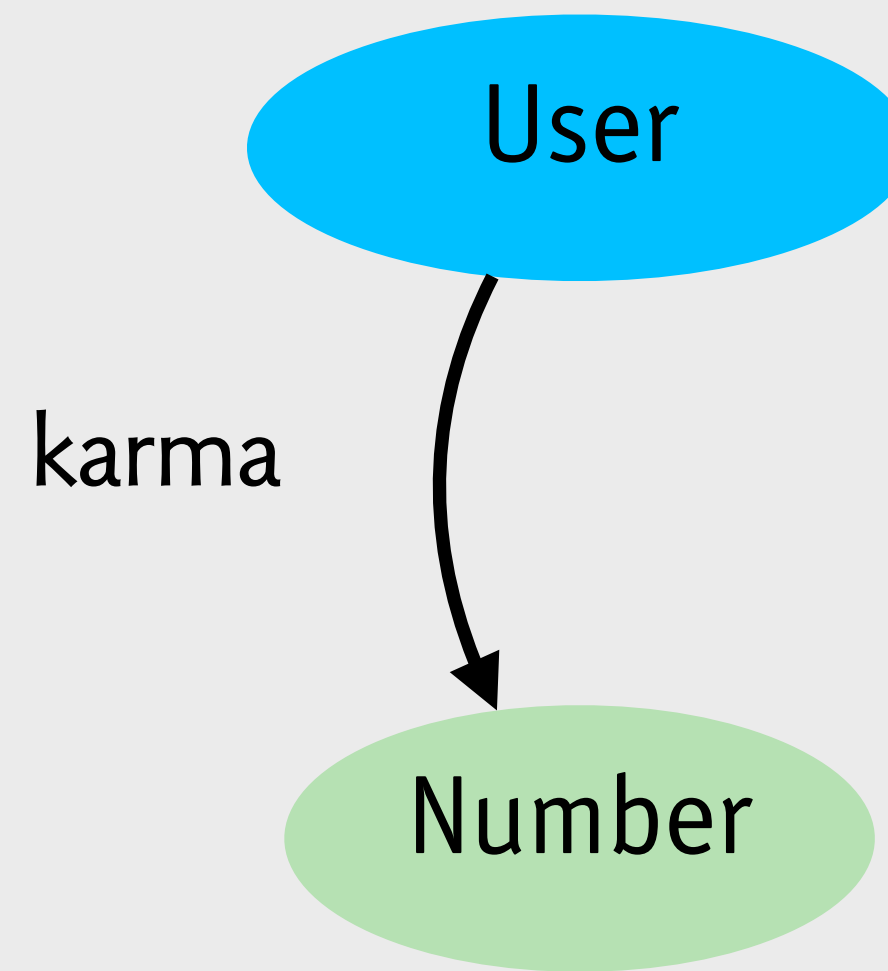


multiple
views

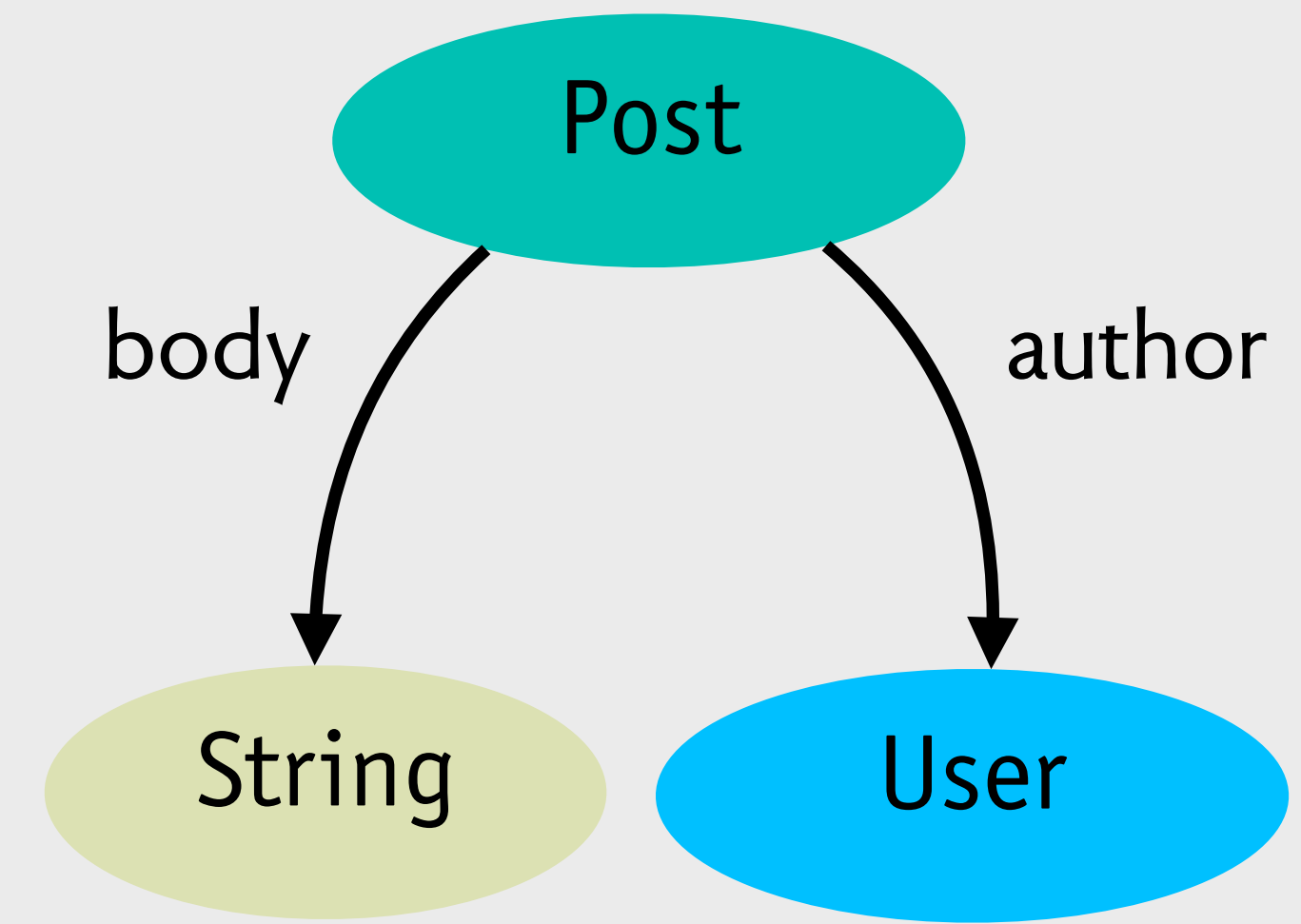
same individual, different properties



concept Authenticating
state
a set of Users with
a username String
a password String

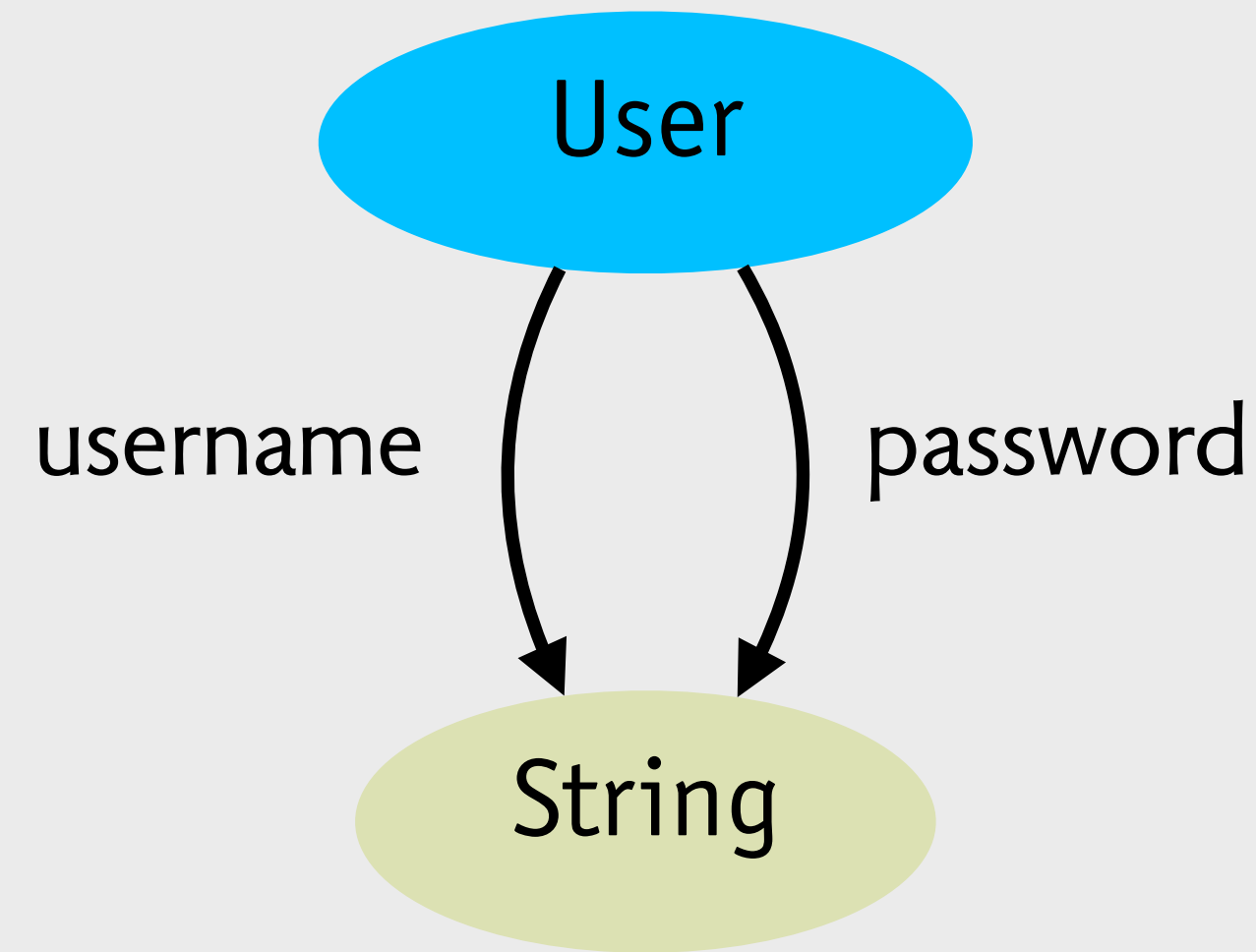


concept KarmaTracking [User]
state
a set of Users with
a karma Number



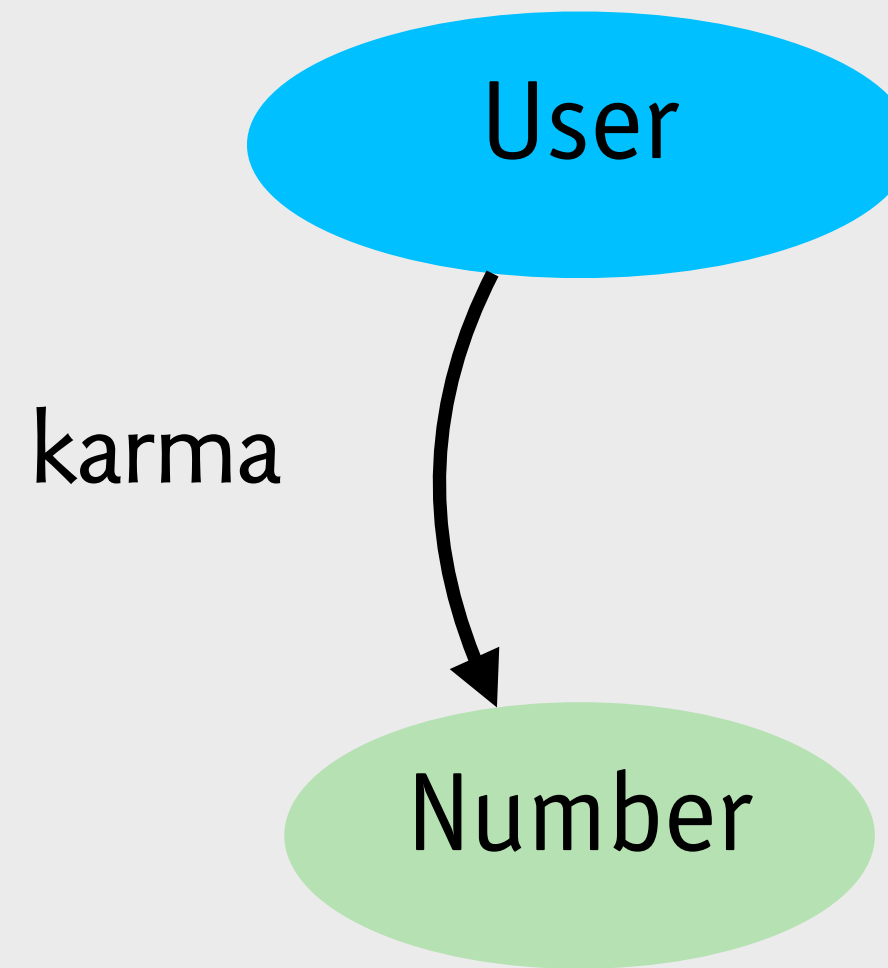
concept Posting [User]
state
a set of Posts with
an author User
a body String

same individual, different properties



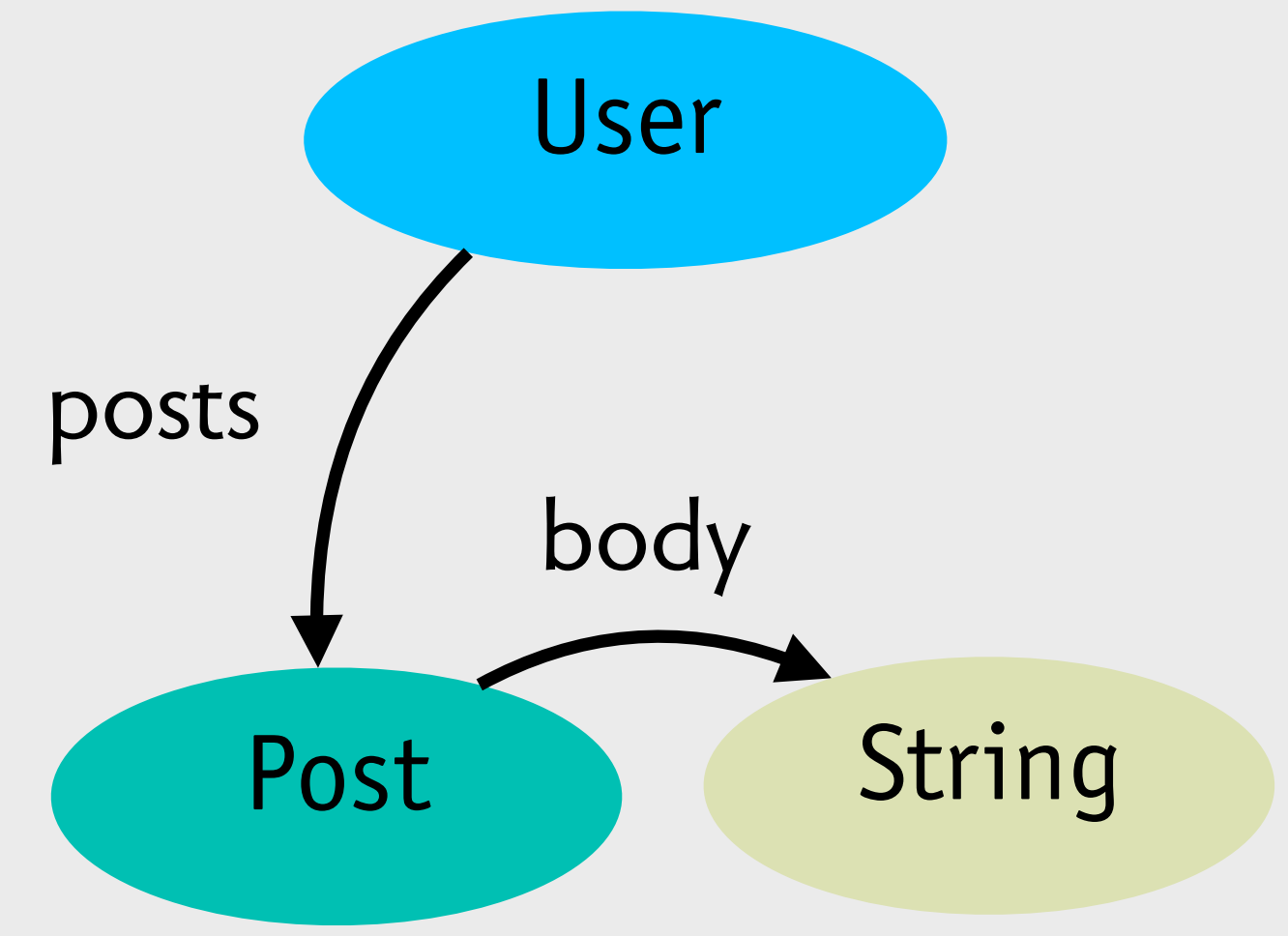
concept Authenticating
state

a set of Users with
a username String
a password String



concept KarmaTracking [User]
state

a set of Users with
a karma Number

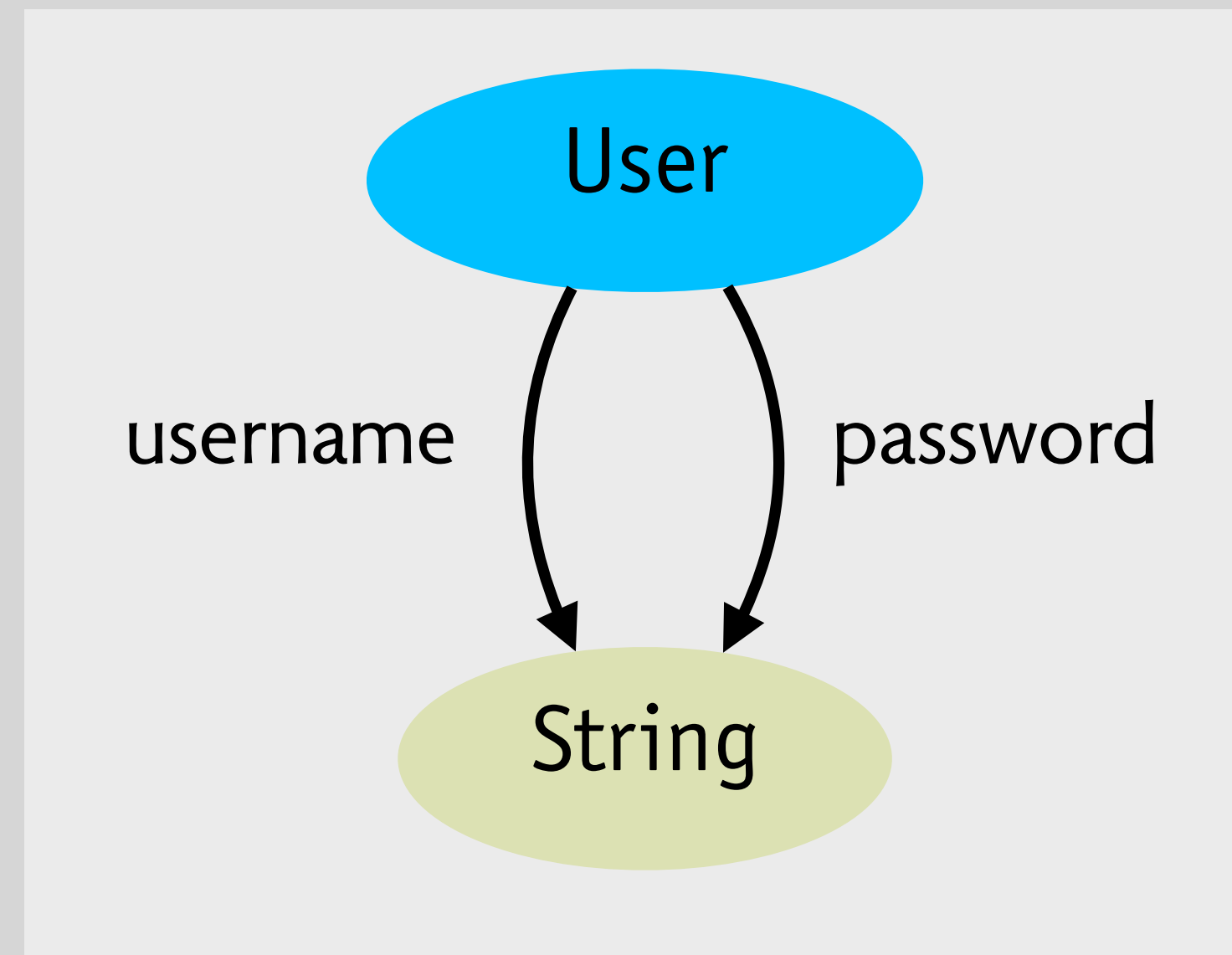


concept Posting [User]
state

a set of Users with
a set of Posts
a set of Posts with
a body String

database
representations

how to represent this state?



concept Authenticating
state

a set of Users with
a username String
a password String

relational database

<i>user</i>	<i>username</i>	<i>password</i>
1	asha	foo
2	bert	whizzy
3	carla	3hg7@!

User table in a relational database

```
CREATE TABLE user ( id INTEGER PRIMARY KEY AUTOINCREMENT,  
username VARCHAR(150) NOT NULL UNIQUE,  
password VARCHAR(128) NOT NULL );
```

```
INSERT INTO user (username, password) VALUES ('asha', 'foo');
```

```
SELECT id, username, password FROM user WHERE username = 'asha';
```

```
UPDATE user SET password = 'bar' WHERE id = 1;
```

```
DELETE FROM user WHERE id = 1;
```

CRUD operations in SQL

document database

<i>_id:</i>	1
<i>username</i>	asha
<i>password</i>	foo

<i>_id:</i>	2
<i>username</i>	bert
<i>password</i>	whizzy

<i>_id:</i>	3
<i>username</i>	carla
<i>password</i>	3hg7@!

CRUD operations in MongoDB

User collection in a document database

```
import { MongoClient, Collection, ObjectId } from 'mongodb';  
// declare document type  
interface UserDocument {  
  _id?: ObjectId;  
  username: string;  
  password: string; }  
// get collection  
const users: Collection<UserDocument> = db.collection<UserDocument>("users");  
// insert document into collection  
await users.insertOne({ username: "asha", password: "foo" });  
// find document in collection (returns regular JavaScript object)  
const user = await users.findOne({ username: "asha" });  
// update document in collection  
await users.updateOne( { username: "asha" }, { $set: { password: "bar" } } );  
// delete document in collection  
await users.deleteOne({ username: "asha" });
```

object-relational mapper

```
from django.db import models
class User(models.Model):
    username = models.CharField(max_length=150, unique=True)
    password = models.CharField(max_length=128)
# creating a new user
new_user = User.objects.create(username="asha", password="foo")
# getting a user by username
try:
    user = User.objects.get(username="alice")
    print(f"Found user: {user.username}")
except User.DoesNotExist:
    print("User not found")
# updating a user
user.password = "bar"
user.save()
# deleting a user
user.delete()
```

*rich state
examples*

playlists

outlining the concept

concept Playlisting [Track]

purpose curate sequences of tracks; prevents having to repeatedly find tracks you want to play

principle after you make a playlist and add tracks to it, you can select a track in the list and can advance to the next track.

why not include playing?

adding state & actions

concept Playlisting [Track]

state

a set of Playlists with
a name String
a set of Tracks
an optional selected Track

actions

make (name: String): (playlist: Playlist)
 then create new playlist with name
add (track: Track, playlist: Playlist)
 where track not in playlist
 then add track to playlist
remove (track: Track, playlist: Playlist)
 where track in playlist
 then remove track from playlist, and
 if selected, set to no longer selected
select (track: Track, playlist: Playlist)
 where track in playlist
 then make track playlist's selected
advance (playlist: Playlist)
 where some track is selected
 then set selected to ???

oops! how do we
fix the state?

introducing track order

concept Playlisting [Track]

state

a set of Playlists with
a name String
a sequence of Tracks
an optional selected Track

actions

make (name: String): (playlist: Playlist)
 then create new playlist with name
add (track: Track, playlist: Playlist)
 where track not in playlist
 then add track to end of sequence
remove (track: Track, playlist: Playlist)
 where track in playlist
 then remove track from playlist, and
 if selected, set to no longer selected
select (track: Track, playlist: Playlist)
 where track in playlist
 then make track playlist's selected
advance (playlist: Playlist)
 where some track is selected that is
 not the last track in the sequence
 then set selected to next in sequence

can a track appear
in two playlists?

can a track appear
twice in one playlist?

can two playlists have
the same songs?

with explicit entries

concept Playlisting [Track]

state

a set of Playlist with
a name String
a set of **Entry**
an optional selected **Entry**
a set of Entry with
a Track
an index Number

actions

make (name: String): (playlist: Playlist)
 then create new playlist with name
add (track: Track, playlist: Playlist)
 ~~**where** track not in playlist~~
 then create new **entry for track**
 and give it index one more than largest
remove (**entry: Entry**, playlist: Playlist)
 where entry in playlist's sequence
 then remove entry, unselect if selected,
 and adjust indexes for other entries
select (**entry: Entry**, playlist: Playlist)
 where entry in playlist's sequence
 then make entry the selected entry
advance (playlist: Playlist)
 where some **entry** is selected that is
 not the playlist entry with highest index
 then set selected to entry with next index

which action is most expensive to perform?

what other popular action will be hard?

what strategies for ordering are there?

on remove, just leave indexes alone

decimals: on insert, compute midpoint

lexorank: index with lex-ordered strings

why “tuple types” like Entry are helpful

concept Playlisting [Track]

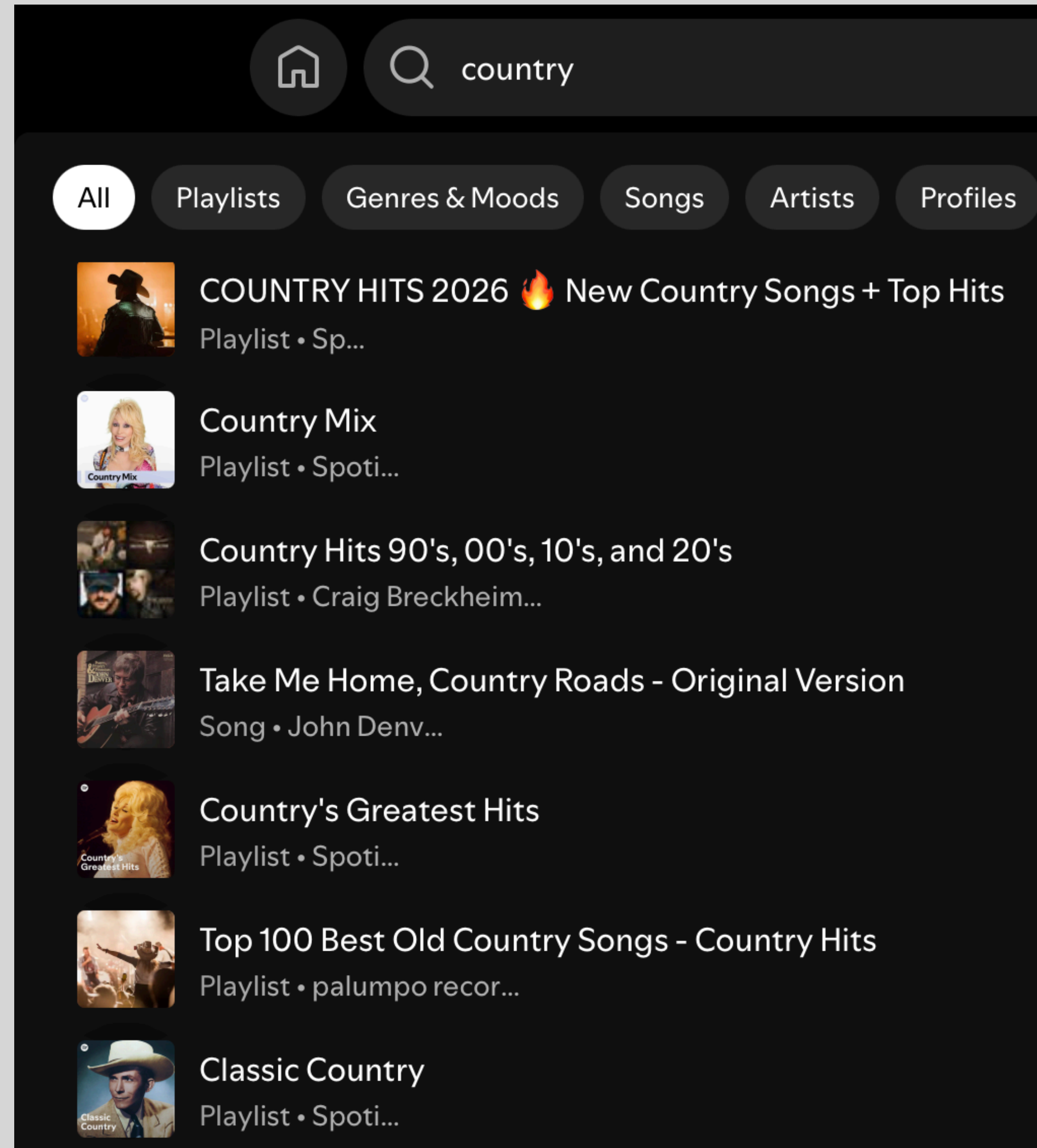
state

a set of Playlist with
 a name String
 a set of Entry
 an optional selected Entry
a set of Entry with
 a Track
an added Date
an index Number

actions

make (name: String): (playlist: Playlist)
 then create new playlist with name
add (track: Track, playlist: Playlist)
 then create new entry for track
 with today's date as added date
 and give it index one more than largest
remove (**entry: Entry**, playlist: Playlist)
 where entry in playlist's sequence
 then remove entry, unselect if selected,
 and adjust indexes for other entries
select (**entry: Entry**, playlist: Playlist)
 where entry in playlist's sequence
 then make entry the selected entry
advance (playlist: Playlist)
 where some **entry** is selected that is
 not the playlist entry with highest index
 then set selected to entry with next index

what is the *real* purpose of playlists?





Public Playlist

COUNTRY HITS 2026

Country Songs + Top Hits

best country music of 2026 most popular trending today modern mix viral top 100...



Spot

• 1,205,820 saves • 114 songs, about 5 hr 30 min



List



#	Title	Album	⌚
---	-------	-------	---

1	 I Can't Love You Anymore Ella Langley, Morgan Wallen	Dandelion	3:48
---	---	-----------	------

2	 Boston STELLA LEFTY	Boston	2:50
---	--	--------	------

3	 Sunburn Tucker Wetmore	Sunburn	3:05
---	---	---------	------

Public

CO

Co

best country music of 2026 most popular trending today modern mix
viral top 100 hottest contemporary country pop rock latest americana
⚠ NO SOCIAL MEDIA ACCOUNTS ASSOCIATED WITH THIS PROFILE.
BEWARE OF SCAMS!



Spot

• 1,205,821 saves • 114 songs, about 5 h

bot-farm account:
included warning
because other scammers
are impersonating *them*!

hugely famous stars

"industry plant"

Public Playlist

Country Mix

Dolly Parton, Bonnie Raitt, The Handsome Family and more

Made for **danieljacksonspotify** • 50 songs, about 2 hr 30 min

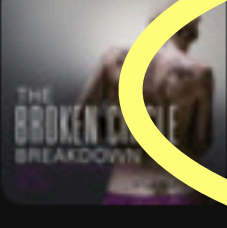
About recommendations and the impact of promotion



List

#	Title	Album	
1	 Light of a Clear Blue Morning Dolly Parton	New Harvest...First Gath...	4:56
2	 Angel from Montgomery - 2008 Re...	Streetlights	3:59

Title

-  Light of a Clear Blue Morning
Dolly Parton
-  Angel from Montgomery - 2008 Re...
Bonnie Raitt
-  Far from Any Road
The Handsome Family
-  If I Needed You
Townes Van Zandt
-  These Hills
Iris DeMent
-  Ou Es-Tu, Mon Amour?
Willie Nelson
-  Dusty Mixed Feelings
Bjorn Eriksson

Public Playlist

Country Mix

Dolly Parton, Bonnie Raitt, The Handsome Family and more

Made for **danieljacksonspotify** • 50 songs, about 2 hr 30 min

About recommendations and the impact of promotion

List

#	Title	Album	
1	 Light of a Clear Blue Morning Dolly Parton	New Harvest...First Gath...	4:56
2	 Angel from Montgomery - 2008 Re... Bonnie Raitt	Streetlights	3:59

About recommendations

In some cases, commercial considerations may influence our recommendations. For example, Spotify offers a promotional tool that enables artists and record labels to highlight priority songs, increasing the likelihood of them being recommended in specific algorithmic playlists: Radio, Autoplay and certain Mixes. This can help artists be discovered by new listeners and grow their fan bases, including prior to a new release, as part of a cultural moment, or in celebration of a milestone. For streams of those priority songs generated in impacted playlists, Spotify charges a commission. This tool is not active in other playlists, such as editorial ones.

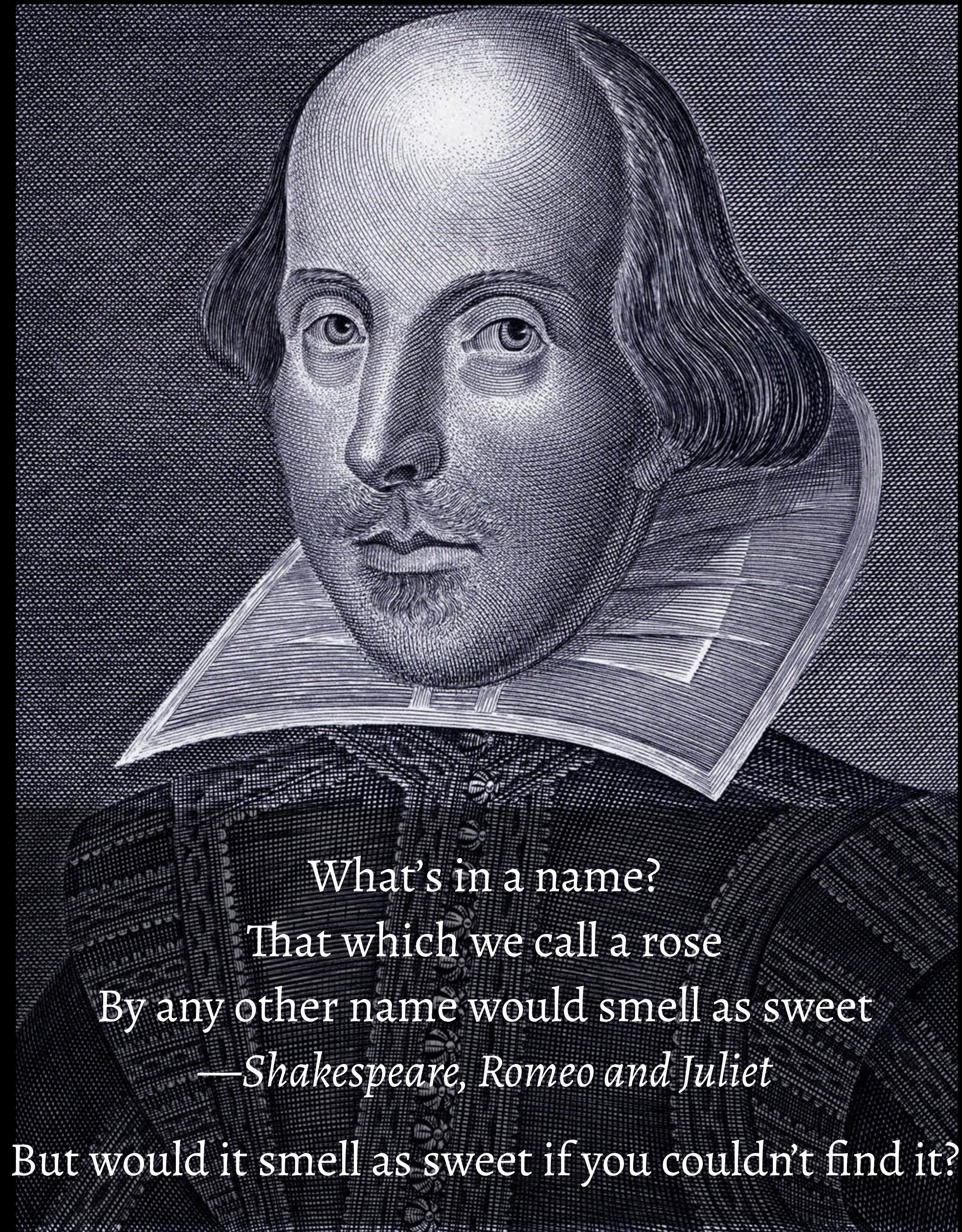
designing enshittification

A feed that is controlled by a company using secret algorithms is much harder for a rival to replicate. That's why Spotify is so hellbent on getting you to listen to playlists, rather than albums. Your favorite albums are the same no matter where you are, but playlists are integrated into services.

But there's another side to this **playlistification** of feeds: playlists and other recommendation algorithms are chokepoints: they are a way to durably interpose a company between a creator and their audience. Where you have chokepoints, you get chokepoint capitalism...

—Cory Doctorow, *Freedom of reach is freedom of speech*

naming



What's in a name?

That which we call a rose

By any other name would smell as sweet

—*Shakespeare, Romeo and Juliet*

But would it smell as sweet if you couldn't find it?

a basic naming concept

concept Naming [Thing]

purpose find things more easily;
prevents having to examine each
thing in detail to find just one

principle after you assign a name
to a thing, you can find it by that
name

state

a named set of Thing with
a name String

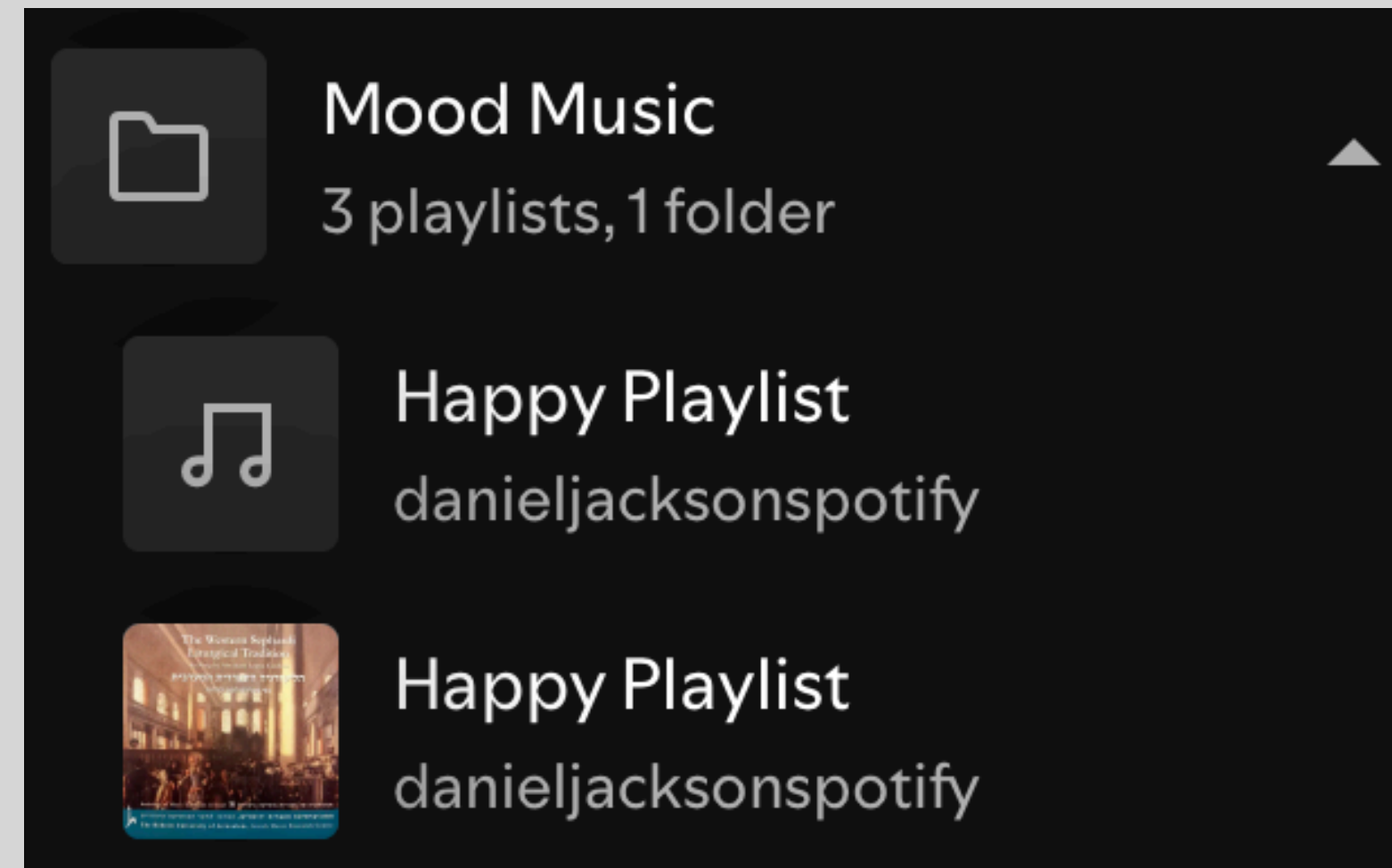
actions

assign (name: String, thing: Thing)

then add thing with given name

_find (name: String): many (thing: set Thing)

return all things with name



naming Spotify playlists

factoring out naming

concept Playlisting [Track]

state

a set of Playlists with
a name String
a set of Tracks
an optional selected Track

when Requesting.request (makePlaylist, name)

then

Playlisting.make (name)

concept Naming [Thing]

state

a named set of Thing with
a name String

concept Playlisting [Track]

state

a set of Playlists with
a set of Tracks
an optional selected Track

when Requesting.request (makePlaylist, name)

then

Playlisting.make (): (playlist)
Naming.assign (thing: playlist, name)

globally unique names

concept GlobalNaming [Thing]

state

a named set of Thing with
a name String

actions

generate (thing: Thing): (name: String)

where no other thing has name

then add thing with name

_find (name: String): optional (thing: Thing)

return thing with name (or none)

1 Global naming

How would you generate globally unique names without coordination?



AVLC8Y

<https://class.mit-sdg.dev/join>

globally unique identifiers

concept GlobalNaming [Thing]

state

a named set of Thing with
a name String

actions

generate (thing: Thing): (name: String)

where no other thing has name

then add thing with name

_find (name: String): optional (thing: Thing)

return thing with name (or none)

how would you
implement generate
without global
coordination?

idea: generate ids
randomly in huge
space (128 bits)

problem: when used
as database keys,
randomness causes
fragmentation

solution: generate
almost in order by
setting first 48 bits
to Unix timestamp

contextual naming

concept ContextualNaming [Namespace, Thing]

state

a set of Namespace with

a set of Entry

a set of Entry with

a name String

a Thing

actions

add (namespace: Namespace, name: String, thing: Thing): (entry: Entry)

where no entry in namespace with name

then create new entry in namespace with name and thing and return it

remove (namespace: Namespace, entry: Entry)

where entry in namespace

then remove entry from namespace

_find (namespace: Namespace, name: String): optional (thing: Thing)

return thing with name for some entry in namespace (or none)

what's a typical invariant?

can you have two entries for the same thing? yes, hence "add" not "assign"

this explains why so remove != delete (eg in Unix)

what if Thing is a Namespace?

recursive lookup

concept ContextualNaming [Namespace, Thing]

state

a set of Namespace with

 a set of Entry

a set of Entry with

 a name String

 a Thing

actions

_find (namespace: Namespace, name: String): optional (thing: Thing)

return thing with name for some entry in namespace (or none)

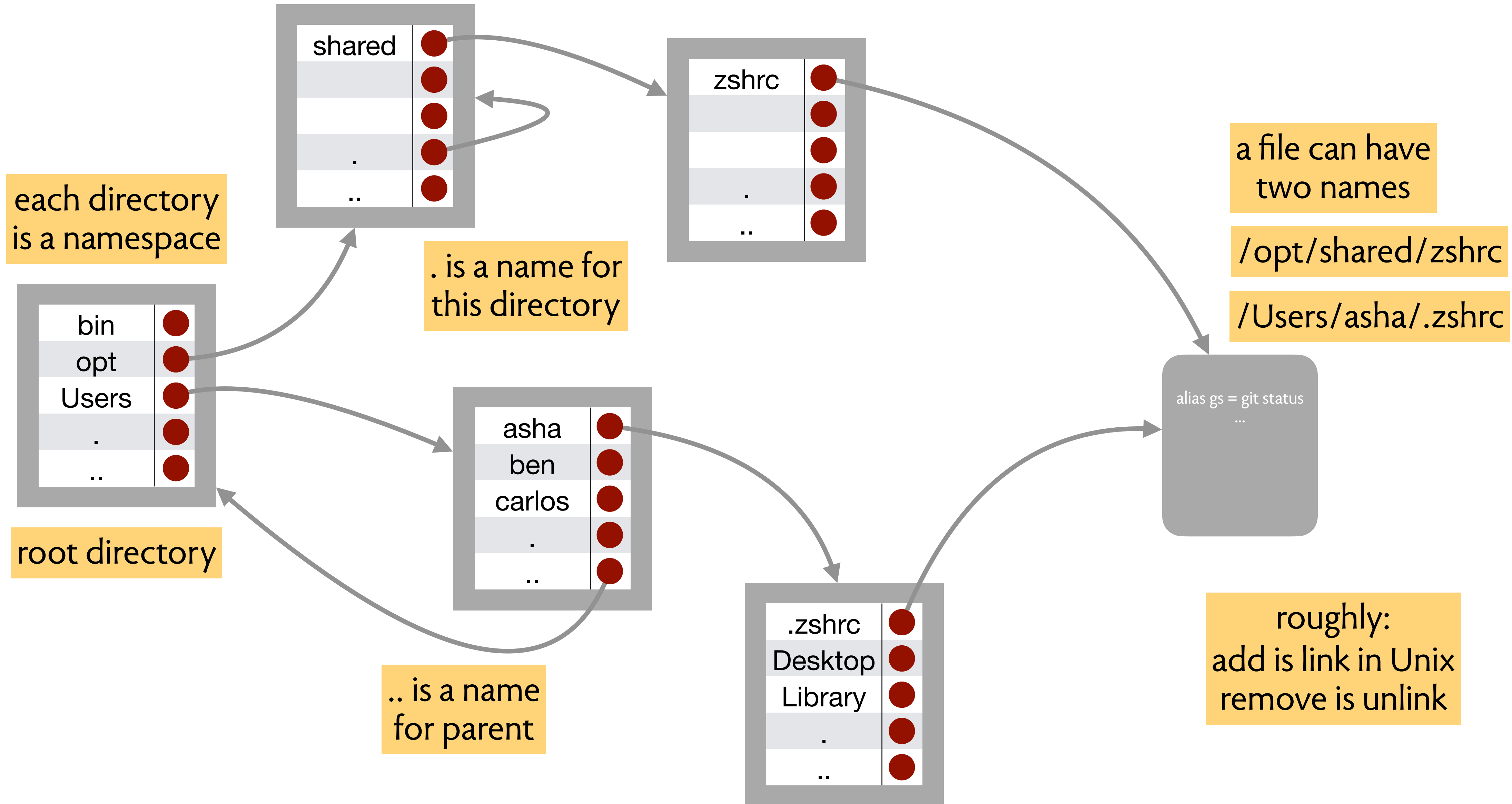
_resolve (namespace: Namespace, path: seq String): optional (thing: Thing)

return

 if path is [n] then _find (namespace, n)

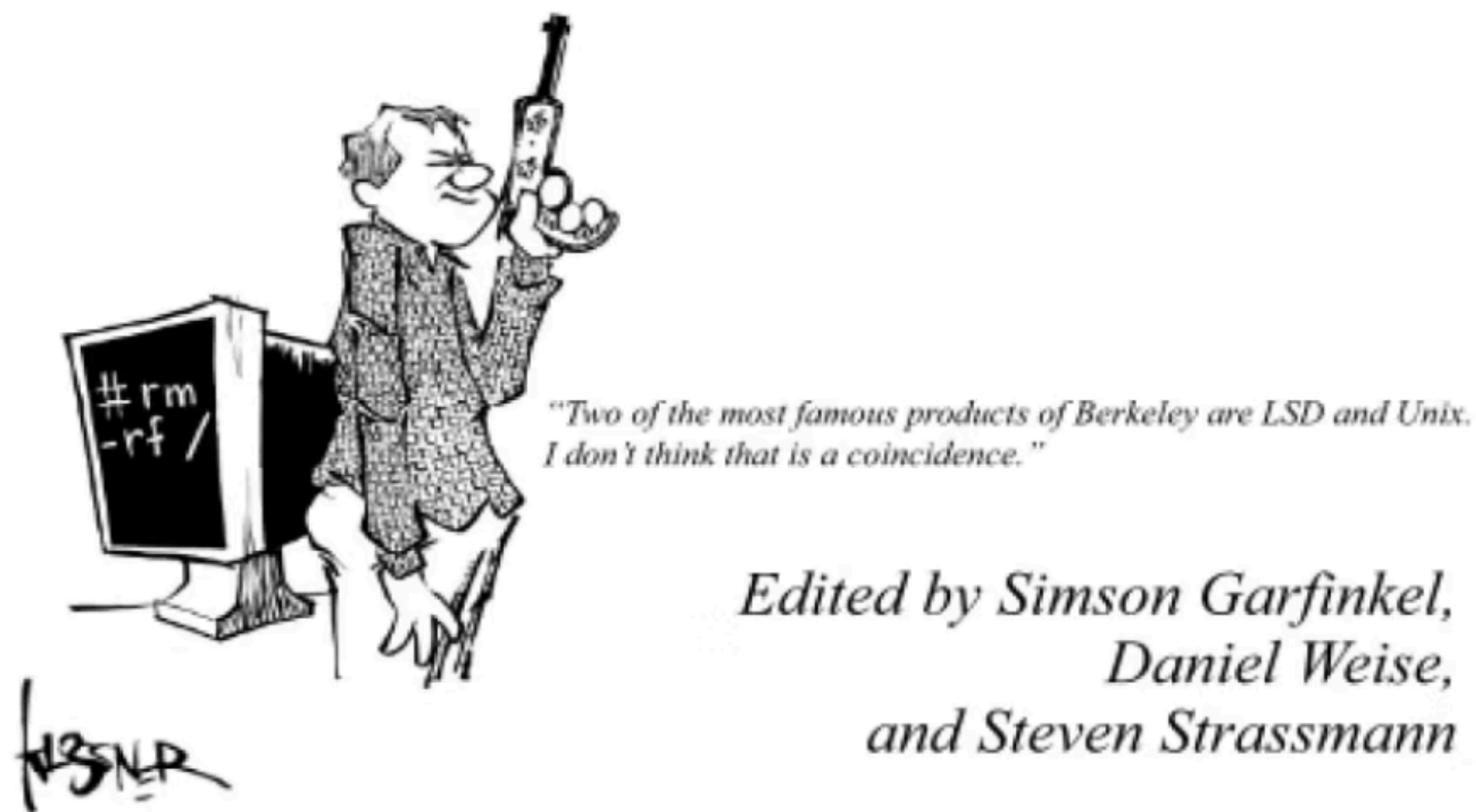
 else if path is [n::rest] then

 _resolve (_find (namespace, n), rest)



the delights of unix

The UNIX- HATERS Handbook



```
mv notes.txt old-notes.txt
// renames notes.txt to old-notes.txt
// but if old-notes.txt exists, overwrites it
```

mv is not a concept
action!

```
mv notes.txt archive/
// moves notes.txt into directory called archive
```

```
mv notes.txt diary.txt archive/
// if you type return too early (before "archive"), then...
```

```
ln notes.txt archive/notes.txt
// add name archive/notes.txt to file called ./notes.txt
```

```
rm notes.txt
// remove name entry for notes.txt
```

```
ls
file1.c, file2.c, file1.o, file2.o
```

```
rm *.o
// remove all files with .o extension
```

```
rm *>o
// remove all files and send command output to new file o
```

example: DNS

a root name server (aka resolver)

NS	edu	edu-server.net
A	edu-server.net	192.5.6.30

a name server for .edu

NS	mit.edu	dns.mit.edu
A	dns.mit.edu	18.0.72.3

a name server for mit.edu

A	mit.edu	23.50.69.136
A	csail.mit.edu	128.52.132.19

order of namespaces

csail.mit.edu as a path is /edu/mit/csail

basic intuition

a top-level namespace resolves edu

a namespace for edu resolves mit

a namespace for mit resolves csail

in practice, a bit trickier

resolver for a namespace has its own domain name

so dns.mit.edu might resolve mit.edu names

how name is resolved

when resolver is asked about csail.mit.edu,

it looks at its entries for resolvers (NS)

for edu, then mit.edu, then csail.mit.edu

if none, it looks at its entries for IP addresses (A)

for csail.mit.edu

zones

don't need a resolver for each domain

dns.mit.edu's could resolve mit.edu and sdg.csail.mit.edu

glue records

how to find a resolver?

it may resolve its own domain name!

so also store resolver's IP address (A)

parting
thought

how to become more tech-savvy

when you're learning a new technology

ask yourself: what concepts are in play?

how do they work? what's clever?

when you're using apps & services

ask yourself: has a concept purpose been hijacked?

whose purpose is being served?